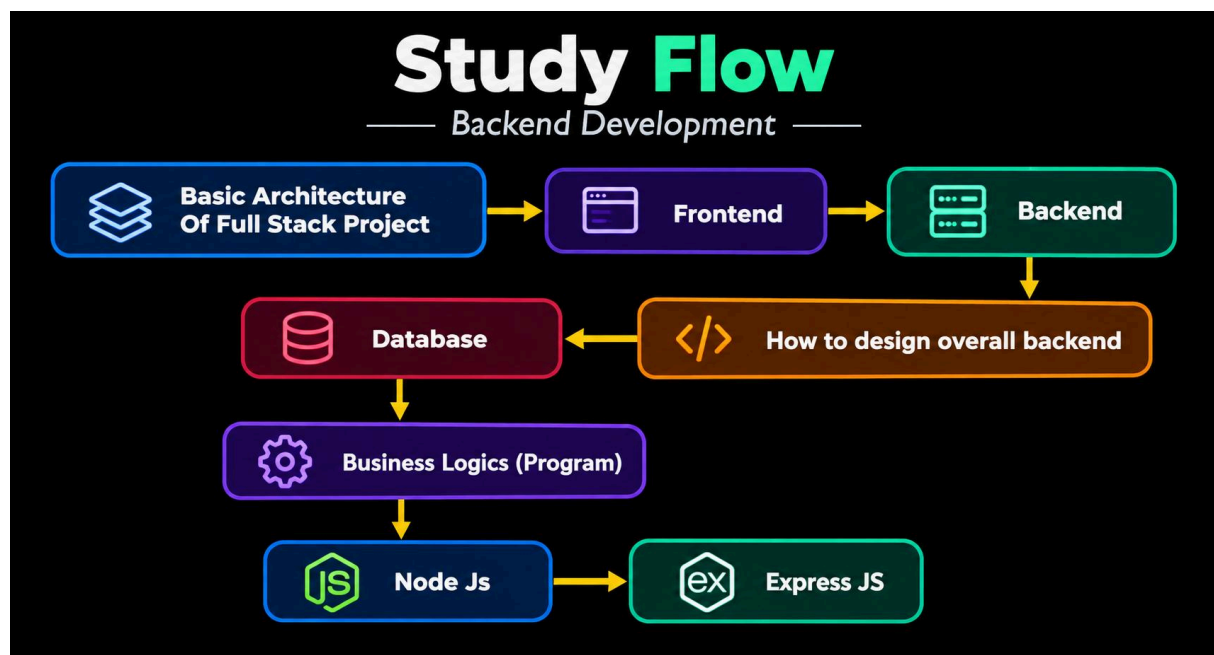


BACKEND DEVELOPMENT

STUDY FLOW

1. Basic Architecture Of Full Stack Project
2. Frontend
3. Backend
4. How to design overall backend
5. Database
6. Business Logics (Program)
7. Node JS
8. Express JS



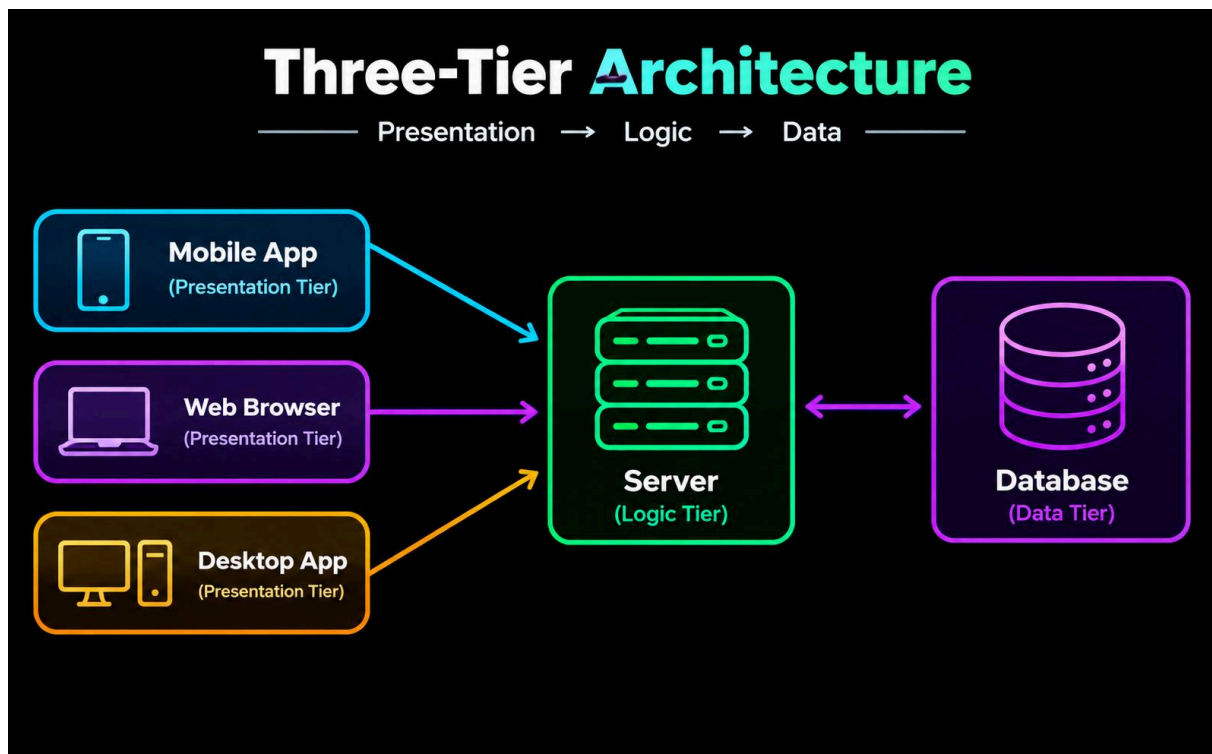
BASIC ARCHITECTURE OF FULL_STACK PROJECT

What Is a Full Stack Project ?

A full-stack application has 3 main layers

- Frontend (Client)
- Backend (Server)
- Database

These 3 talk to each other in a fixed workflow.



Frontend (Client Side)

The **Frontend (Client Side)** is the part of a **web application** that runs in the user's **browser**. In this project, the frontend is built using **React.js in the MERN stack** and is mainly responsible for providing a smooth **UI** and **UX**.

What Frontend Does

- Shows UI (buttons, forms, pages)
- Takes user input (login, signup, post, comment)
- Sends data to backend via HTTP requests
- Displays response from backend

Backend (Server Side)

The **Backend (Server Side)** is the part of a **web application** that runs on the **server**. In this project, the backend is built using **Node.js and Express.js in the MERN stack** and acts as the brain of the application, handling requests, processing data, and communicating with the **database**.

What Backend Does

- Receives request from frontend
- Validates data
- Applies business logic
- Talks to database
- Sends response back to frontend

Backend Responsibilities

- Authentication (login / signup)
- Authorization (who can do what)
- Validation
- Security
- APIs
- Data processing

Database (Data Layer)

The **Database (Data Layer)** is the part of an **application** that stores data permanently. In the **MERN stack, MongoDB** is used as the database to **store application** and **user data** in a structured manner.

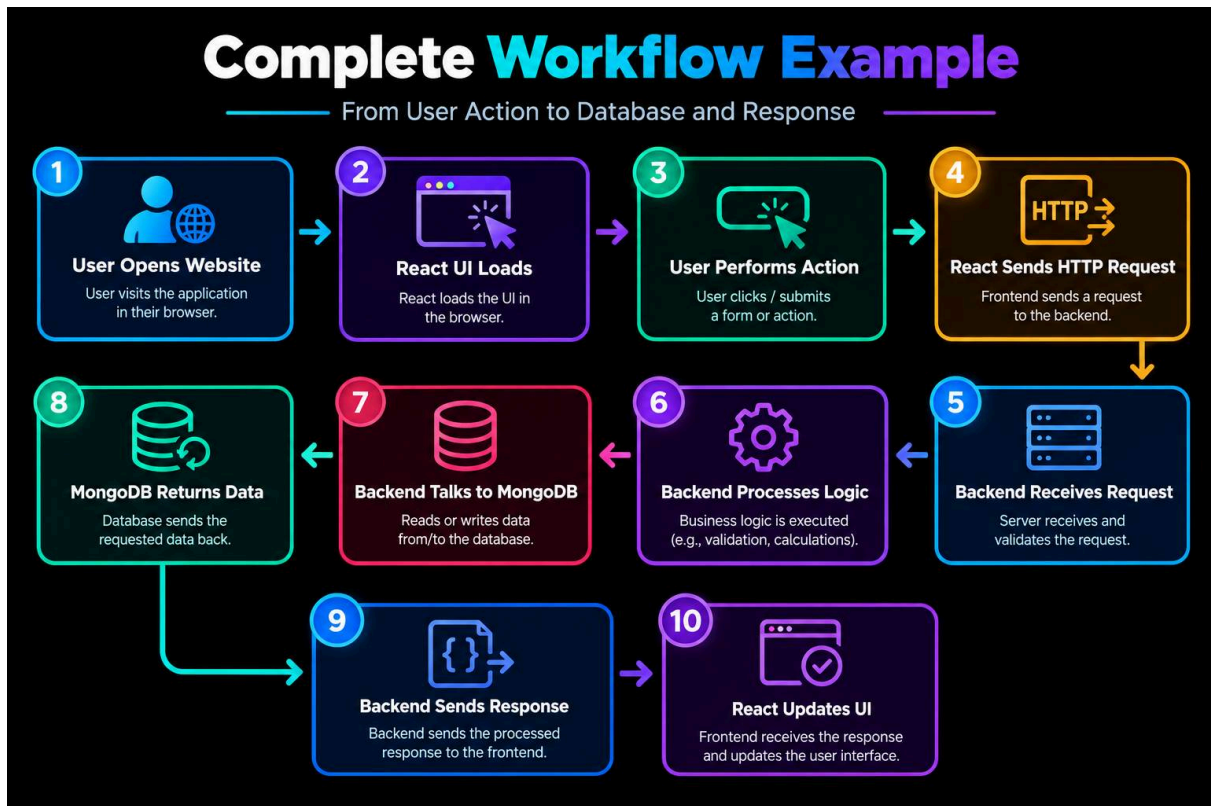
What Database Does

- Stores user data in a structured manner.

Database Responsibilities

- Data Definition
- Data Storage
- Data Retrieval

- Data Manipulation
- Data Integrity
- Concurrency Control
- Security & Access Control



COMPLETE BACKEND CRASH COURSE



Node.js = runtime environment ✓

```

JavaScript (language)
↓
Node.js (runtime)
↓
Express (framework)
  
```

Difference Between Node and Express

Node.js (http)	Express.js
Manual routing	Built-in routing
Add headers manually	Headers are auto handled
More code	Less code
Hard to scale	Easy to scale
Low-level	High-level

Node.js makes it possible to create a server, while **Express.js** makes server development more human-friendly and easier to manage.

Create a Basic Server

Installation Of Express

```
npm install express
```

Express JS Code To Create a Basic Server

```
import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send([
    { id: 100, name: "hussain", age: 21 },
    { id: 101, name: "suleman", age: 22 },
    { id: 102, name: "zaryab", age: 20 },
  ]);
});

app.listen(7000, () => {
  console.log("server is running on http://localhost:7000");
});
```



Since you are using **ES6** imports so you have to set module type **"type":** **"module"** manually in your **package.json** file

Nodemon Development Tool For Node JS

nodemon is a development tool for Node.js that automatically restarts the server whenever you make changes to your code.

Using **nodemon** as an **npm** Script

We can use **nodemon** as an **npm** script so that we can start our Node/Express server with a simple command **npm run dev**.

Install **nodemon**

```
npm install --save-dev nodemon
```

Add **nodemon** to **package.json**

```
"scripts": {  
  "dev": "nodemon server.js"  
}
```

Here

- **dev** → the name of our custom **npm** script.
- **nodemon server.js** → the command that will be executed.
- **server.js** → our main server file.

Run the script

Now instead of writing

```
npx nodemon server.js
```

we can simply use

```
npm run dev
```

Why use Express instead of Core Node JS ?

`Express.js` is a web framework built on top of `Node.js`. It makes creating web servers and APIs easier, shorter, and more organized. If you're building a **web server/API**, `Express.js` is used on top of core `Node.js` mainly because it **removes** a lot of **repetitive work**.

Server Using Core Node JS

```
import http from "http";

const server = http.createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Hello from Node.js server");
  } else if (req.url === "/about" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("About page");
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page not found");
  }
});

server.listen(7000, () => {
  console.log("Node.js server running on http://localhost:7000");
});
```

You have to manually check

```
req.url
req.method
res.writeHead()
res.end()
```

Server Using Express JS

```
import express from "express";

const app = express();
```

```

app.get("/", (req, res) => {
  res.send("Hello from Express server 🚀");
});

app.get("/about", (req, res) => {
  res.send("About page");
});

app.listen(7000, () => {
  console.log("Express server running on http://localhost:7000");
});

```

Much cleaner.

The important idea



Express does NOT replace Node JS.

Think of it like

```

Node.js
  ↓
HTTP Server + Runtime
  ↓
Express
  ↓
Routing + Middleware + Request/Response helpers
  ↓
Your API / Web Application

```

So, you use core **Node JS** when you want low-level control or you're learning how HTTP servers work. You use **Express** when you want to build APIs and web applications with less boilerplate and a structured development pattern.

Postman vs Desktop Agent

Postman is a tool used to test APIs. You can send requests such as **GET**, **POST**, **PUT**, and **DELETE** to your backend and check the responses.

Postman Desktop Agent is a small application that runs on your computer and allows the **Postman web app to communicate with services running on your local machine**, such as `localhost`.

In simple words

- **Postman** → Used to create and test API requests.
- **Desktop Agent** → Connects the Postman web app to your local computer/server.
- **Postman + Desktop Agent** → You can easily test a local backend/API from Postman.

Postman

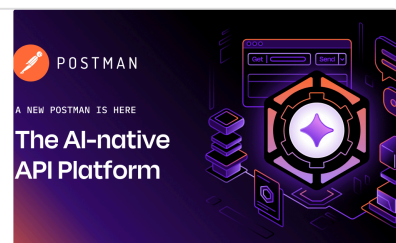
Postman is an API development and testing tool. It is used to send requests such as **GET, POST, PUT, and DELETE** to a backend/server and check the response. For example, if your backend is running on `localhost:5000`, you can test an API like `http://localhost:5000/api/users` in Postman. Postman is available as both a **desktop app and a web app**.

Official Postman website

Postman: The World's Leading API Platform | Sign Up for Free

Accelerate API development with Postman's all-in-one platform. Streamline collaboration and simplify the API lifecycle for faster, better results. Learn more.

 <https://www.postman.com/>



Postman Desktop Agent

Postman Desktop Agent is a small application that runs on your computer. It is mainly used when you are using **Postman in the browser** and want to send requests to **local/private APIs**, such as a backend running on `localhost`. It helps the Postman web app communicate with your local machine and overcome browser **CORS limitations**.

Official Desktop Agent Download

Postman Agent: For Mac, Windows, & Linux

The Postman agent overcomes the Cross-Origin Resource Sharing (CORS) limitations of browsers, and facilitates API request sending from your browser version of Postman.

 <https://www.postman.com/downloads/postman-agent/>



Request & Response Cycle

Backend Code

```
import express from "express";

const app = express();

// Middleware: allows Express to read JSON data
app.use(express.json());

app.post("/create-user", (req, res) => {
  // Receive data from request
  const { name, email } = req.body;
  console.log("Received:", req.body);

  // Send response back
  res.status(201).json({
    message: `Welcome ${name}!`,
    email: email
  });
});

app.listen(7000, () => {
  console.log("Server running on http://localhost:7000");
});
```

Sending Data from Postman

In Postman, select
method

POST

URL

`http://localhost:7000/create-user`

Go to

Body → raw → JSON

Then send

```
{
  "name": "hussain",
  "email": "hussainbscs2024@gmail.com"
}
```

Postman sends this JSON as the request body to the backend.

What Happens in Backend ?

The request reaches

```
app.post("/create-user", (req, res) => {
  // Receive data from request
  const { name, email } = req.body;
  console.log("Received:", req.body);

  // Send response back
  res.status(201).json({
    message: `Welcome ${name}!`,
    email: email
  });
});
```

Express matches

POST + /create-user

Then

```
req.body
```

contains

```
{  
  "name": "hussain",  
  "email": "hussainbscs2024@gmail.com"  
}
```

We can extract the values

```
const { name, email } = req.body;
```

Now

```
name → hussain  
email → hussainbscs2024@gmail.com
```

Backend Sends Response

The backend sends

```
res.status(201).json({  
  message: `Welcome ${name}!`,  
  email: email  
});
```

Postman receives

```
{  
  "message": "Welcome hussain!",  
  "email": "hussainbscs2024@gmail.com"  
}
```

Middleware

Middleware is a function that runs **between the incoming request and the final response**. It can inspect, modify, or process the request before it reaches the route handler.

For example

```
app.use(express.json());
```

This middleware allows Express to read and parse JSON data sent in the request body, so we can access it using

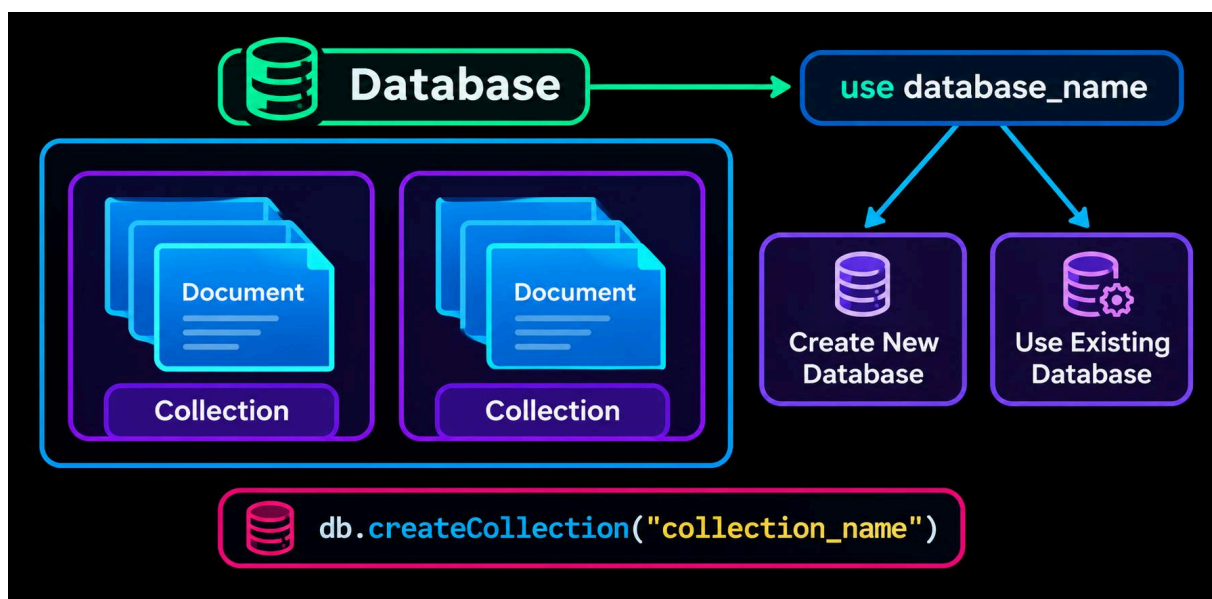
```
req.body
```

Database (MongoDB)

MongoDB is a NoSQL, document-oriented database. Instead of storing data in tables and rows like MySQL, it stores data in **collections and documents**.

Basic Structure

```
Database
  ↓
Collection
  ↓
Document
  ↓
Fields
```



Example document

```
{
  "name": "Hussain",
  "age": 21,
  "city": "Lahore",
  "skills": ["JavaScript", "React"]
}
```

MongoDB v/s SQL

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
SQL queries	MongoDB Query Language
JOIN	\$ Lookup / embedded documents

MongoDB Basic Operations with Mongoose

```
Database    → mongoose.connect()
Collection → mongoose.model()
Document    → Model.create()
```

Example

MongoDB concept	Code line
Database → Connect	<code>mongoose.connect("mongodb://127.0.0.1:27017/myDatabase")</code>
Collection → Model	<code>const User = mongoose.model("User", userSchema)</code>
Document → Create	<code>User.create({ name: "Hussain", age: 21 })</code>

MongoDB Atlas

MongoDB Atlas is a **cloud-based service** provided by MongoDB that allows us to create, store, and manage MongoDB databases online.

MongoDB Atlas Hierarchy



Organization → Project → Cluster → Database → Collection → Document

Level	Meaning
Organization	Highest-level container; manage teams/projects
Project	Organize related Atlas resources
Cluster	MongoDB database environment where data is stored
Database	Group of related collections
Collection	Group of related documents
Document	Actual data/object

```
Organization
  ↓
Project
  ↓
Cluster
  ↓
Database
  ↓
Collection
  ↓
Document
```

Easy example

```
Organization: My Company
  ↓
Project: E-Commerce
  ↓
Cluster: E-Commerce Cluster
  ↓
Database: shopDB
  ↓
Collection: users
  ↓
Document:
{
  name: "Ali",
  age: 22
}
```

Mongoose

Mongoose is a **Node.js library (ODM)** used to work with MongoDB more easily.

Basic Flow

```
Node.js
  ↓
Mongoose
  ↓
MongoDB Atlas
  ↓
Cluster
  ↓
Database
  ↓
Collection
  ↓
Documents
```

Installation

```
npm install mongoose
```

Important Code Lines

Purpose	Code
Import	<code>const mongoose = require("mongoose")</code>
Connect	<code>mongoose.connect("MONGODB_URL")</code>
Create Schema	<code>const userSchema = new mongoose.Schema({...})</code>
Create Model	<code>const User = mongoose.model("User", userSchema)</code>
Create Document	<code>User.create({...})</code>
Read	<code>User.find()</code>
Update	<code>User.updateOne({...}, {\$set: {...}})</code>
Delete	<code>User.deleteOne({...})</code>

MongoDB Cluster Connectivity with backend

Syntax

```
mongoose.connect(URL)
```

For Example

```
(async () => {
  const connectionInstance = await mongoose.connect(
    "mongodb+srv://hussainbscs2024_db_user:fVaS62sjtawGIh7T@cluster0.i7g0l4r.mongodb.net/",
  );

  console.log(connectionInstance.connection.host);
})();
```

Setup Schema

Syntax

```
mongoose.schema({...})
```

For Example

```
const userSchema = mongoose.Schema({
  name: String,
  age: Number,
});
```

Creating a Collection

syntax

```
const userCollection = mongoose.model(name, schema);
```

For Example

```
const userCollection = mongoose.model("User", userSchema);
```

Insert Document

Syntax

```
collectionName.create(documentObj)
```

For Example

```
const createdUser = userCollection.create(userObj);
```

CRUD Operations

Create

```
app.post("/create-user", async (req, res) => {  
  const userObj = req.body;  
  const createdUser = await userCollection.create(userObj);  
  res.send({ createdUser: createdUser });  
});
```

Read

```
app.get("/get-users", async (req, res) => {  
  const users = await userCollection.find();  
  res.send(users);  
});
```

Read Single Item

```
app.get("/get-single-user", async (req, res) => {  
  const { name } = req.body;  
  const user = await userCollection.findOne({ name: name });  
  res.send(user);  
});
```

Update

```
app.put("/update-user", async (req, res) => {  
  const userId = req.query.id; // postman main params se aye  
  gi userId  
  const newUser = req.body; // postman main body se aye ga n  
  ewUser  
  const updatedUser = await userCollection.findByIdAndUpdate  
  (userId, newUser, {  
    new: true,
```

```
});  
res.send({ updatedUser: updatedUser });  
});
```

Delete

```
app.delete("/delete-user", async (req, res) => {  
  const userId = req.query.id;  
  const deletedUser = await userCollection.findByIdAndDelete  
(userId);  
  res.send({  
    deletedUser: deletedUser,  
  });  
});
```